with similar data living closer together. The labels A, B, C, and D point to particular clusters of words that share some semantics.

Figure 1.7. In the RL framework, some kind of learning algorithm decides which actions to take for a control task (e.g., driving a robot vacuum), and the action results in a positive or negative reward, which will positively or negatively reinforce that action and hence train the learning algorithm.

Figure 1.8. The standard framework for RL algorithms. The agent takes an action in the environment, such as moving a chess piece, which then updates the state of the environment. For every action it takes, it receives a reward (e.g., +1 for winning the game, –1 for losing the game, 0 otherwise). The RL algorithm repeats this process with the objective of maximizing rewards in the long term, and it eventually learns how the environment works.

Figure 1.9. The input data (which is the state of the environment at some point in time) is fed into the agent (implemented as a deep neural network in this book), which then evaluates that data in order to take an action. The process is a little more involved than shown here, but this captures the essence.

Figure 1.10. The OpenAI Python library comes with many environments and an easy-to-use interface for a learning algorithm to interact with. With just a few lines of code, we've loaded up a car racing game.

Figure 1.11. DeepMind's DQN algorithm successfully learned how to play seven Atari games with only the raw pixels as input and the player's score as the objective to maximize. Previous algorithms, such as IBM's Deep Blue, needed to be fine-tuned to play a specific game.

Figure 1.12. An action lookup table for Tic-Tac-Toe with only three entries, where the "player" (an algorithm) plays X. When the player is given a board position, the lookup table dictates the move that they should make next. There will be an entry for every possible state in the game.

Figure 1.13. A series of three frames of Breakout. The placement of the ball is slightly different in each frame. If you were using a lookup table, this would equate to storing three unique entries in the table. A lookup